DQN Implementations: CleanRL Reference and Mixed Backprop Comparison

Technion NeuroAI Lab

March 8, 2026

1 DQN algorithm (CleanRL reference)

The reference implementation follows the CleanRL DQN recipe (see `cleanrl/dqn.py` and our `run_cleanrl_vs_mixed_dqn.py`). The agent interacts with the environment (e.g. CartPole-v1), stores transitions (s, a, r, s', d) in a replay buffer of fixed size, and after a warm-up period (`learning_starts` steps) performs a training step every `train_frequency` steps. Action selection is ϵ-greedy with a *linear schedule* from `start_e` to `end_e` over the first half of total steps (exploration phase).

Architecture. The Q-network is a 3-layer MLP:

$$\mathbf{x} \longrightarrow \mathbb{R}^{120} \xrightarrow{\text{ReLU}} \mathbb{R}^{84} \xrightarrow{\text{ReLU}} \mathbb{R}^{A}, \tag{1}$$

so $Q(s, a; \theta)$ is the a-th output of this network, with $A = n_{\text{actions}}$.

TD target and loss. Let θ^- denote the target network parameters. The TD target is

$$y = r + \gamma\,(1 - d)\,\max_{a'} Q(s', a'; \theta^-). \tag{2}$$

The loss is the mean squared error between the target and the current Q-value for the taken action on a minibatch $\mathcal{B}$:

$$\mathcal{L} = \mathbb{E}_{(s,a,r,s',d)\in\mathcal{B}}\big[\big(y - Q(s, a; \theta)\big)^2\big]. \tag{3}$$

Parameters θ are updated by Adam on $\mathcal{L}$. The target network is updated every `target_network_frequency` steps (we use 500); with $\tau = 1$ this is a full copy $\theta^- \leftarrow \theta$.

2 Other implementations in the same codebase

The script `composable_feedback_cleanrl.py` implements five alternatives; only the differences are summarized here.

Option (a) — Scalar DQN. Identical to the reference in algorithm and network layout (same 120–84–ReLU architecture and single scalar TD error).

Option (b) — Outcome-specific multi-head. Shared trunk and K heads; each head k predicts $Q_k(s, a)$ for an outcome $o^{(k)}$. TD target per head: $y_k = o_k + \gamma\,(1 - d)\,\max_{a'} Q_k^-(s', a')$; loss is MSE between $[Q_1, \ldots, Q_K]$ and $[y_1, \ldots, y_K]$.

Options (c), (c_local), (d) — Feature decomposition. Feature vector $h(s) \in \mathbb{R}^F$, $Q_i(s, a) = h_i(s)\,W_{i,a}$, $Q(s, a) = \sum_i Q_i(s, a) + b_a$. Per-feature target $y_i = r_i + \gamma\,(1 - d)\,Q_i^-(s', a^*)$ with $r_i = r/F$. **(c)** single scalar loss on $\sum_i Q_i$; **(c_local)** $\mathcal{L} = \mathbb{E}[\sum_i \delta_i^2]$ with $\delta_i = y_i - Q_i(s, a)$; **(d)** per-feature MSE.

3 Comparison with the two-layer notebook DQN

The notebooks in `2layers_dqns` use a two-layer linear MLP $Q = W_2(W_1 s + b_1) + b_2$ (or ReLU/LeakyReLU in the hidden layer), same Bellman target and MSE loss. Training runs every step after the buffer is sufficiently full; target network is updated every 500 *episodes*. So compared to CleanRL: shallower net, no `learning_starts`/`train_frequency`, and episode-based target sync.

4 Learned linear mixing of vectorial TD errors (extension)

We extend the two-layer linear DQN so that each feature's learning signal is a *learned linear combination* of the full vectorial TD error. Setup: $h(s) = W_1 s + b_1 \in \mathbb{R}^F$, $Q(s,a) = \sum_{i=1}^{F} Q_i(s,a) + b_{2,a}$ with $Q_i(s,a) = (W_2)_{a,i}\, h_i(s)$. Per-feature TD target $y_i = r_i + \gamma\,(1-d)\, Q_i^-(s', a^*)$ and vectorial error $\boldsymbol{\delta} = (y_1 - Q_1, \ldots, y_F - Q_F)^\top \in \mathbb{R}^F$. Learned mixing matrix $C \in \mathbb{R}^{F \times F}$:

$$e_i = (C\boldsymbol{\delta})_i = \sum_{j=1}^{F} C_{i,j}\, \delta_j, \qquad \mathbf{e} = C\boldsymbol{\delta}. \tag{4}$$

Loss $\mathcal{L} = \|\mathbf{e}\|^2$; all parameters (W_1, b_1, W_2, b_2, C) updated by gradient descent. With $C = I$ this reduces to per-feature MSE; learned C allows each feature to learn from a distinct combination of error components. Figures 3–5 show results from `run_vectorial_dqn.py`.

5 Mixed backprop: nonlinear forward, linear sparse feedback

We extend the architecture so that the *forward* pass is nonlinear (linear features plus a nonlinear trunk), but the *learning signal* for the linear-feature layer is a fixed sparse linear function of the TD error at the Q-output, with no backpropagation through the trunk for that layer. We give two variants: simple mixed ($B \in \mathbb{R}^{F \times A}$) and mixed 3F ($B \in \mathbb{R}^{F \times K}$ with $K = 3F$).

5.1 Shared setup

For direct comparison with CleanRL we use the same architectural dimensions. Let $F = 120$ and $H = 84$. The network has three stages:

1. **Linear features:** $\mathbf{z} = W_z \mathbf{x} + b_z \in \mathbb{R}^F$.
2. **Nonlinear trunk:** $\mathbf{h} = \mathrm{ReLU}(W_h^{(1)} \mathbf{z} + b_h^{(1)}) \in \mathbb{R}^H$, then readout to logits in $\mathbb{R}^A$ (or $\mathbb{R}^K$ for the 3F variant).
3. **Q-output:** $Q(s,a)$ is the a-th component of the readout (or, in the 3F variant, the mean of a group of components; see below).

The TD target is

$$y = r + \gamma\,(1-d)\, \max_{a'} Q^-(s', a'). \tag{5}$$

The *trunk and readout* are always trained by standard backprop on the scalar TD loss

$$\mathcal{L} = \big(y - Q(s,a)\big)^2. \tag{6}$$

The *linear-feature layer* (W_z, b_z) is updated by a separate rule that does not use gradients through the trunk.

5.2 Simple mixed: feedback matrix $B \in \mathbb{R}^{F\times A}$

The readout has dimension A. Define the error vector $\mathbf{e} \in \mathbb{R}^A$ by setting the component for the taken action to the TD error and the others to zero:

$$e_a = y - Q(s,a), \qquad e_{a'} = 0 \ \text{ for } \ a' \neq a. \tag{7}$$

Let $B \in \mathbb{R}^{F\times A}$ be a *fixed* sparse matrix: row i has a single non-zero entry at column $i \bmod A$, with value 1. Thus

$$(B)_{i,\, i \bmod A} = 1, \qquad \text{all other entries } 0. \tag{8}$$

The learning signal for the linear-feature layer is

$$\boldsymbol{\delta}_z = B\mathbf{e} \in \mathbb{R}^F. \tag{9}$$

So feature i receives only the error component $e_{i \bmod A}$. The update for W_z uses no gradient through the trunk: we use the gradient estimate

$$\nabla_{W_z} \leftarrow \frac{1}{|\mathcal{B}|} \sum_{(s,a,\ldots)\in\mathcal{B}} \boldsymbol{\delta}_z^\top \mathbf{x}, \qquad \nabla_{b_z} \leftarrow \frac{1}{|\mathcal{B}|} \sum_{\mathcal{B}} \boldsymbol{\delta}_z, \tag{10}$$

then apply gradient ascent with learning rate η_z (we use $\eta_z = 10^{-4}$), and optionally clip the gradient of W_z to maximum norm 1 for stability. So

$$W_z \leftarrow W_z + \eta_z \nabla_{W_z}, \qquad b_z \leftarrow b_z + \eta_z \nabla_{b_z}. \tag{11}$$

For the simple mixed variant we update the target network every 250 steps (more frequent than DQN's 500) to stabilize learning.

5.3 Mixed 3F: higher-dimensional error and $B \in \mathbb{R}^{F\times K}$

To give each feature a learning signal that combines multiple error channels, we increase the readout dimension to $K = 3F$ (e.g. $K = 360$ when $F = 120$). The trunk outputs $\mathbf{q}_{\text{raw}} \in \mathbb{R}^K$. Partition the channel indices into A groups of size $G = K/A$ (assume K divisible by A): for action $a \in \{0, \ldots, A-1\}$,

$$\mathcal{G}_a = \big\{aG,\, aG+1,\, \ldots,\, (a+1)G-1\big\}. \tag{12}$$

Define the Q-value for action a as the mean over that group:

$$Q(s,a) = \frac{1}{G} \sum_{j\in\mathcal{G}_a} (\mathbf{q}_{\text{raw}})_j. \tag{13}$$

So each action's Q-value is the average of G scalar outputs. The TD target y is unchanged. For the taken action a, define the error vector $\mathbf{e} \in \mathbb{R}^K$ by spreading the scalar TD error uniformly over the channels in $\mathcal{G}_a$:

$$e_j = \frac{y - Q(s,a)}{G} \ \text{ for } \ j \in \mathcal{G}_a, \qquad e_j = 0 \ \text{ otherwise}. \tag{14}$$

Thus $\sum_{j\in\mathcal{G}_a} e_j = y - Q(s,a)$. The fixed feedback matrix $B \in \mathbb{R}^{F\times K}$ is chosen so that each feature i receives the average of exactly three error components: those at indices $3i$, $3i+1$, $3i+2$. So

$$(B)_{i,\,3i} = (B)_{i,\,3i+1} = (B)_{i,\,3i+2} = \frac{1}{3}, \qquad \text{all other entries } 0. \tag{15}$$

Then

$$\delta_{z,i} = (B\mathbf{e})_i = \frac{1}{3}\big(e_{3i} + e_{3i+1} + e_{3i+2}\big). \tag{16}$$

The update for W_z and b_z is the same as in the simple mixed case: $\nabla_{W_z} = \frac{1}{|\mathcal{B}|}\sum \boldsymbol{\delta}_z^\top \mathbf{x}$, then $W_z \leftarrow W_z + \eta_z \nabla_{W_z}$ (with optional gradient norm clipping), and similarly for b_z. The trunk is trained by backprop on $\mathcal{L} = (y - Q(s,a))^2$.

5.4 Algorithm (per step, after sampling a minibatch)

1. Compute TD target $y = r + \gamma(1-d)\max_{a'} Q^-(s',a')$ using the target network.
2. **DQN:** Forward pass with current network; $\mathcal{L} = (y - Q(s,a))^2$; backprop $\mathcal{L}$ and update all parameters.
3. **Mixed / Mixed 3F:** Forward pass with $\mathbf{z}$ computed; feed $\mathbf{z}$.detach() into the trunk so the trunk does not receive gradients from the linear layer. Compute $Q(s,a)$ (and for 3F, $\mathbf{q}_{\text{raw}}$ and group means). Build $\mathbf{e}$ as above (dimension A or K). Compute $\boldsymbol{\delta}_z = B\mathbf{e}$. Form ∇_{W_z} and ∇_{b_z}; optionally clip ∇_{W_z} to max norm 1; update W_z and b_z with learning rate η_z. Then backprop $\mathcal{L} = (y - Q(s,a))^2$ through the trunk only and update trunk parameters with Adam (learning rate 2.5×10^{-4}).
4. Update target network every 500 steps (DQN) or 250 steps (mixed).

We implement CleanRL DQN, simple mixed, and mixed 3F in `run_cleanrl_vs_mixed_dqn.py`.

6 Experimental setup

All comparison runs use the same environment and step-based training loop for fairness.

Environment. CartPole-v1 (Gymnasium).

Main comparison (CleanRL vs Mixed 3F). Total steps: 500 000. Seeds: 1 and 2. Replay buffer size 10 000; batch size 128; training starts after 10 000 steps and every 10 steps thereafter. Discount $\gamma = 0.99$. ϵ-greedy: ϵ decreases linearly from 1 to 0.05 over the first 250 000 steps (exploration phase), then remains 0.05. Learning rate for the Q-network (or trunk): 2.5×10^{-4} (Adam). For mixed variants: linear-feature learning rate $\eta_z = 10^{-4}$, gradient norm clip 1 for W_z; target network updated every 250 steps. Architecture: $F = 120$, $H = 84$, $A = 2$ (CartPole); for mixed 3F, $K = 3F = 360$ and $G = 180$ per action.

7 Results

CleanRL DQN vs Mixed 3F. We ran both algorithms for 500 000 steps with two seeds (Table 1). Figure 1 shows episodic return over training (mean and individual seeds). Figure 2 shows the mean TD loss per episode. Table 2 summarizes final return and mean return over the last 100 episodes. CleanRL DQN solves CartPole (return 500) for both seeds within 500k steps. Mixed 3F solves for one seed and reaches high return (275) for the other, demonstrating that the sparse fixed feedback can support learning while showing higher variance across seeds.

Table 1: Hyperparameters for the CleanRL vs Mixed 3F comparison.

Parameter	Value
Total timesteps	500 000
Seeds	1, 2
Buffer size	10 000
Batch size	128
Learning starts	10 000
Train frequency	10
Target network (DQN)	Every 500 steps
Target network (mixed)	Every 250 steps
γ	0.99
ϵ schedule	Linear $1 \to 0.05$ over 250k steps
Learning rate (trunk/Adam)	2.5×10^{-4}
η_z (mixed linear layer)	10^{-4}
Gradient clip (W_z)	1

Table 2: Results: final return and mean return over last 100 episodes (500k steps, 2 seeds).

Algo	Seed	Final return	Mean last 100
CleanRL DQN	1	500	500
CleanRL DQN	2	500	500
Mixed 3F	1	275	257.4
Mixed 3F	2	500	500

Vectorial and simple mixed (earlier experiments). Figures 3–5 show the vectorial-error DQN (learned C) vs baseline two-layer linear DQN from `run_vectorial_dqn.py`. Figures 6–8 show the simple mixed-backprop DQN (fixed $B \in \mathbb{R}^{F \times A}$) vs baseline nonlinear DQN from `run_mixed_backprop_dqn.py`.

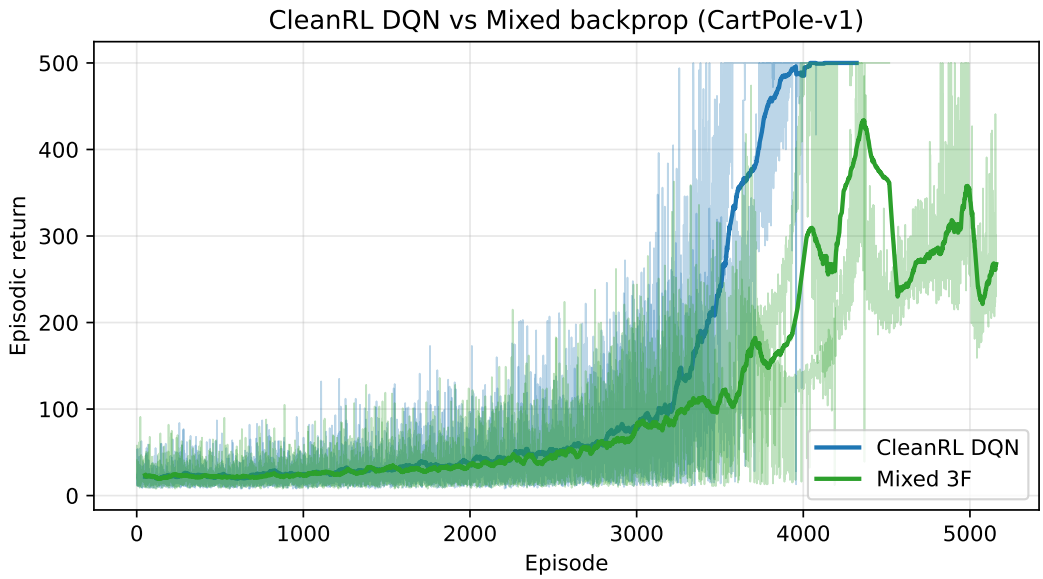

Figure 1: Episodic return over training. CleanRL DQN and Mixed 3F on CartPole-v1 (500k steps, 2 seeds).

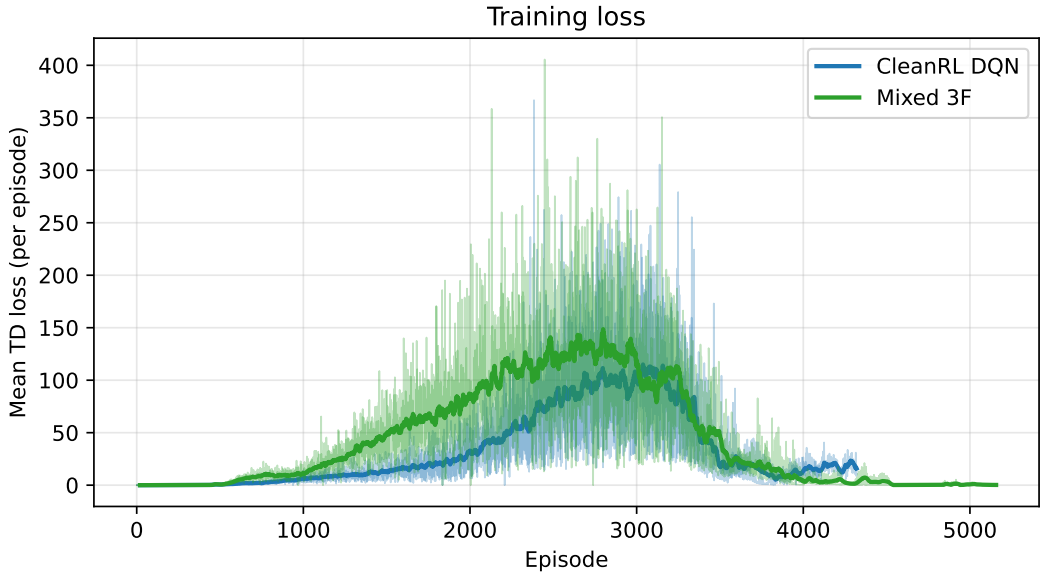

Figure 2: Mean TD loss per episode. CleanRL DQN and Mixed 3F.

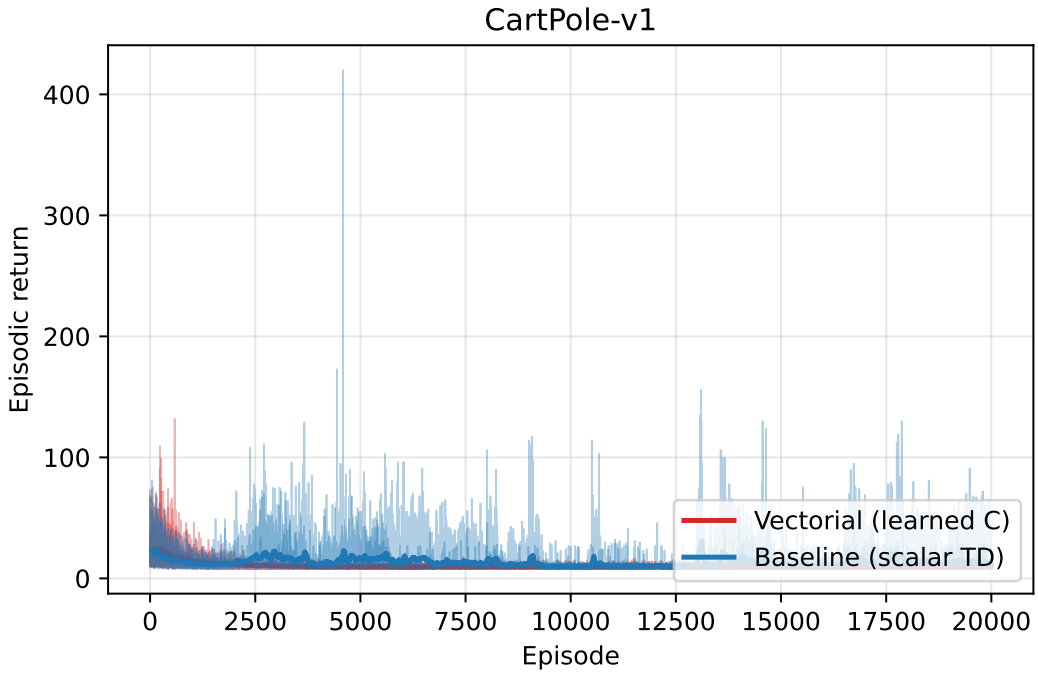

Figure 3: Episodic return. Vectorial-error DQN (learned C) and baseline two-layer linear DQN on CartPole-v1.

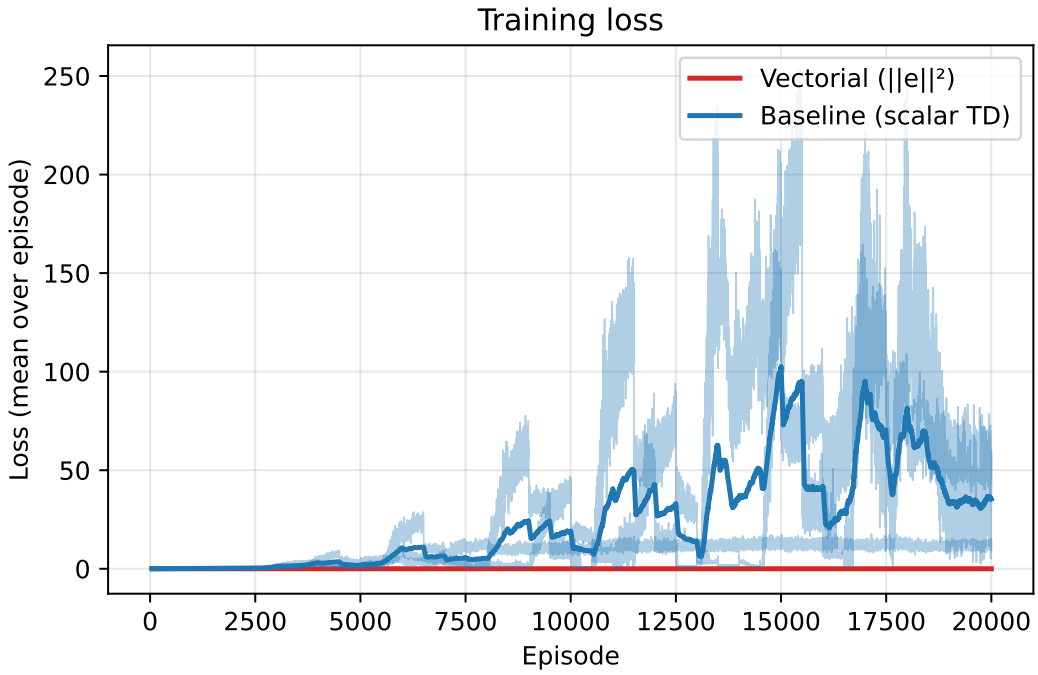

Figure 4: Training loss. Baseline: scalar TD MSE. Vectorial: $\|\mathbf{e}\|^2$.

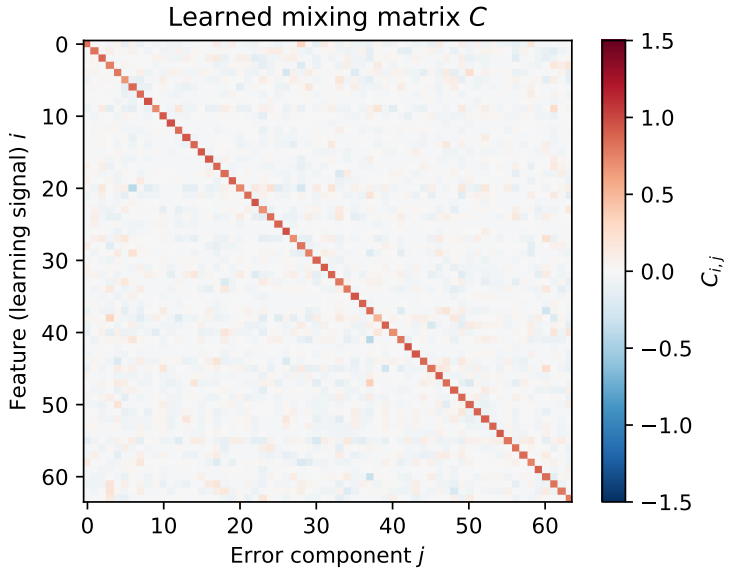

Figure 5: Learned mixing matrix C at end of training (vectorial run).

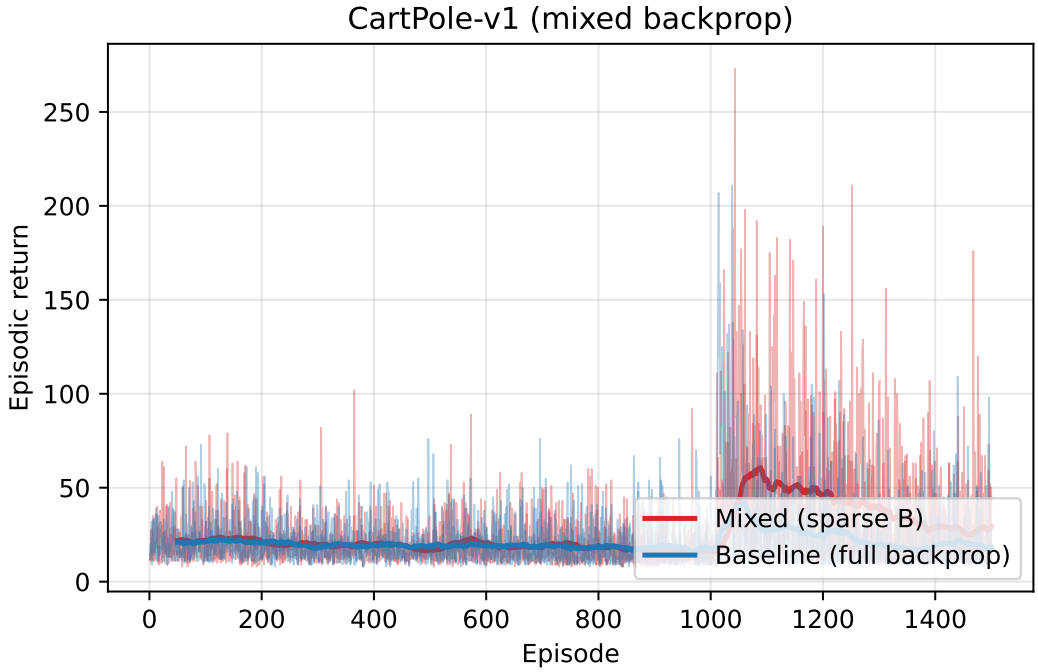

Figure 6: Episodic return. Mixed-backprop DQN (fixed $B \in \mathbb{R}^{F \times A}$) and baseline nonlinear DQN.

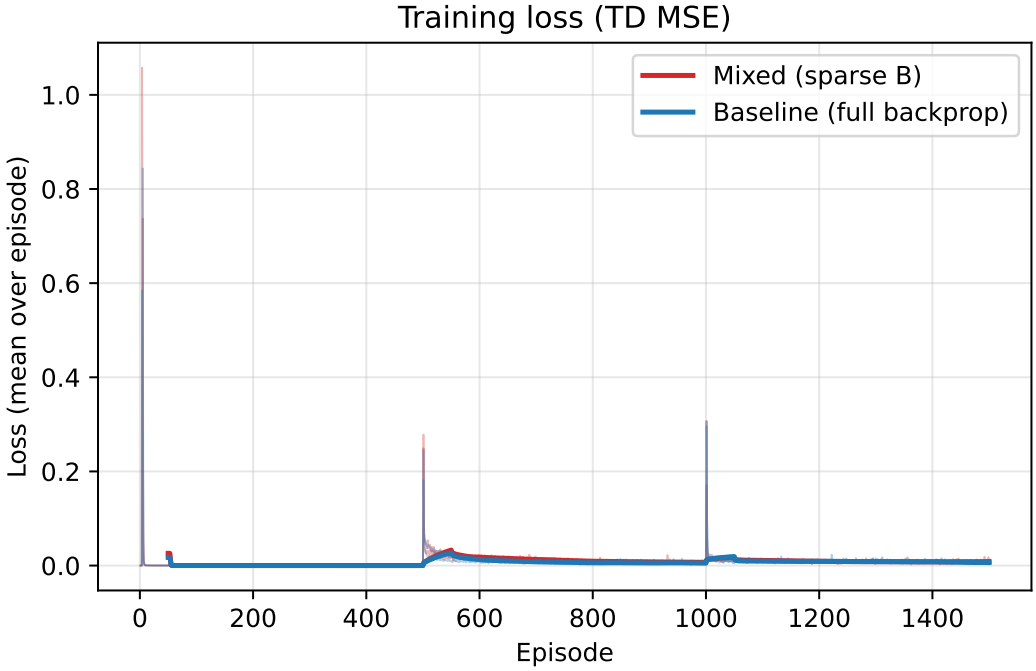

Figure 7: Training loss. Mixed-backprop and baseline nonlinear DQN.

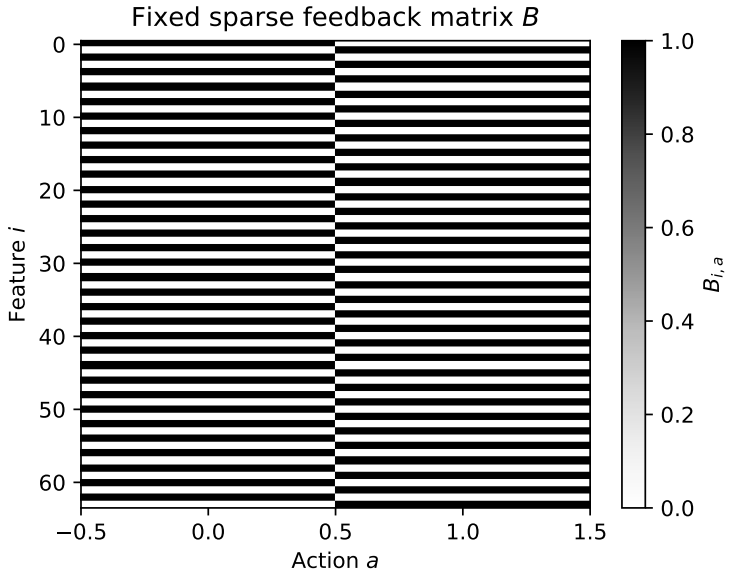

Figure 8: Fixed sparse feedback matrix $B \in \mathbb{R}^{F \times A}$: row i gives the combination of action errors for $\delta_{z,i}$.